Análise do impacto de estruturas de dados otimizadas na complexidade assintótica e no desempenho prático de algoritmos

Athos Marques[1], **Bruna C. Markowisk**[2], **Thiago Branco**[3]

[1] Departamento de Engenharia de Software e Sistemas de Informação
Pontifícia Universidade Católica de Minas Gerais (PUC Minas) – Belo Horizonte, MG – Brazil

{bcmarkowisk,amrfonseca,thiago.branco}@sga.pucminas.br

Abstract. *This study investigates the impact of optimized data structures on algorithm performance, focusing on the Disjoint Set Union (DSU). Through an experimental approach, three variants were implemented and compared: Naive, Union by Rank, and Full Tarjan, using execution time and memory access metrics. The results demonstrate the transition from quadratic complexity to near-constant efficiency $O(\alpha(n))$ through Robert Tarjan's heuristics. A stable 33% reduction in operations was observed with path compression, validating the importance of proper data structure selection for the scalability of real-world systems, such as Kruskal's algorithm.*

Resumo. *Este trabalho investiga a influência de estruturas de dados otimizadas no desempenho prático de algoritmos, utilizando como objeto de estudo a estrutura Disjoint Set Union (DSU). Através de uma abordagem experimental, compararam-se as variantes Naive, Union by Rank e Full Tarjan, mensurando tempo de execução e acessos à memória. Os resultados evidenciam a inviabilidade da implementação ingênua em larga escala e a eficiência das heurísticas de união por posto e compressão de caminho, que convergem para uma redução de 33% nas operações. Conclui-se que a complexidade amortizada $O(\alpha(n))$ é fundamental para a escalabilidade de algoritmos reais, como o de Kruskal.*

1. Introdução

A análise de algoritmos constitui uma etapa fundamental no desenvolvimento de soluções computacionais eficientes, pois permite estimar o comportamento de uma implementação à medida que o tamanho da entrada aumenta. Nesse contexto, a complexidade assintótica fornece uma base teórica para comparar algoritmos independentemente de características específicas de hardware, linguagem de programação ou ambiente de execução. Entretanto, em aplicações práticas, o desempenho observado também pode ser influenciado por fatores concretos de implementação, como a escolha das estruturas de dados, o padrão de acesso à memória e as otimizações realizadas pelo ambiente de execução.

Entre as estruturas de dados utilizadas para resolver problemas de conectividade dinâmica, destaca-se a estrutura *Disjoint Set Union* (DSU), também conhecida como *Union-Find*. Essa estrutura permite representar uma coleção de conjuntos disjuntos e executar operações de criação de conjuntos, busca de representantes e união entre conjuntos. Tais operações são especialmente relevantes em algoritmos clássicos sobre grafos, como o algoritmo de Kruskal, no qual é necessário verificar de forma eficiente se dois

vértices pertencem ao mesmo componente conexo antes de adicionar uma aresta à árvore geradora mínima.

Apesar de sua simplicidade conceitual, a eficiência da estrutura DSU depende diretamente das estratégias utilizadas em sua implementação. Uma versão ingênua, sem heurísticas de otimização, pode produzir árvores altamente desbalanceadas, elevando o custo das operações de busca. Por outro lado, técnicas como *Union by Rank* e compressão de caminho, associadas ao algoritmo de Tarjan, reduzem significativamente a profundidade das árvores e melhoram o desempenho amortizado das operações.

Diante disso, este trabalho analisa o impacto de estruturas de dados otimizadas na complexidade assintótica e no desempenho prático de algoritmos. Para isso, são implementadas e comparadas três variantes da estrutura DSU: uma versão ingênua, uma versão com união por posto e uma versão completa com união por posto e compressão de caminho. A comparação considera tanto a fundamentação teórica das complexidades esperadas quanto os resultados experimentais obtidos por meio da medição do tempo de execução e da contagem de acessos à memória em diferentes tamanhos de entrada.

A relevância deste estudo está na possibilidade de demonstrar, de forma teórica e empírica, como escolhas aparentemente internas à implementação de uma estrutura de dados podem alterar significativamente a escalabilidade de um algoritmo. Assim, busca-se evidenciar que a seleção adequada de estruturas e heurísticas de otimização não apenas melhora o desempenho em cenários específicos, mas também pode tornar viável a execução de algoritmos em entradas de grande escala.

2. Objetivos

O objetivo geral deste trabalho é analisar o impacto do uso de heurísticas de otimização na estrutura de dados *Disjoint Set Union*, considerando tanto a complexidade assintótica das operações quanto o desempenho prático observado em experimentos computacionais.

Como objetivos específicos, este trabalho busca:

- apresentar os conceitos fundamentais da estrutura DSU e suas operações principais, `makeSet`, `findSet` e `union`;
- implementar três variantes da estrutura DSU: uma versão ingênua, uma versão com *Union by Rank* e uma versão com *Union by Rank* associada à compressão de caminho;
- comparar teoricamente as complexidades assintóticas das variantes implementadas, destacando os efeitos das heurísticas de balanceamento e compressão;
- definir um protocolo experimental capaz de medir o tempo de execução e a quantidade de acessos à memória realizados por cada variante;
- avaliar empiricamente o comportamento das implementações para diferentes tamanhos de entrada;
- verificar a relação entre os resultados experimentais obtidos e as complexidades teóricas esperadas para cada abordagem.

3. Metodologia

3.1. Implementação das Variantes

A organização do projeto foi estruturada segundo o princípio da separação de responsabilidades, de modo que cada componente do sistema possui uma função bem delimitada e independente. Essa escolha visou facilitar tanto a extensibilidade da solução quanto a confiabilidade dos experimentos comparativos. Adicionalmente, a implementação do sistema foi realizada integralmente em Java.

A classe central da arquitetura é a classe abstrata `DSU`, que formaliza o contrato das três operações fundamentais descritas por Cormen et al. [Cormen et al. 2009]: `makeSet(x)`, `findSet(x)` e `union(x, y)`. A opção por classe abstrata, em detrimento de uma interface, justifica-se pelo fato de que parte do estado e do comportamento é comum a todas as variantes. O vetor `parent[]` que armazena a estrutura de parentesco da floresta, e o contador de acessos à memória (`operationCount`), utilizado na análise experimental, são atributos compartilhados por todas as implementações. Centralizá-los na classe abstrata garante que as três variantes sejam medidas sob as mesmas condições, eliminando possíveis vieses na coleta de dados. Diante do exposto, é possível observar, no Código 1, a implementação da classe abstrata `DSU`.

Código 1: Classe Abstrata `DSU`

```java
public abstract class DSU {
    protected int[] parent;
    protected int n;

    protected long operationCount;

    public DSU(int n) {
        this.n = n;
        this.parent = new int[n];
        this.operationCount = 0;
        for (int i = 0; i < n; i++) {
            makeSet(i);
        }
    }

    public abstract void makeSet(int x);

    public abstract int findSet(int x);

    public abstract void union(int x, int y);

    public boolean isItInTheSameSet(int x, int y) {
        return findSet(x) == findSet(y);
    }

    public long getOperationCount() {
        return operationCount;
    }

    public void resetOperationCount() {
        operationCount = 0;
    }
}
```

As três variantes concretas, `DSUNaive`, `DSUUnionByRank` e `DSUFullTarjan`, derivam dessa classe abstrata e implementam apenas os comportamentos específicos que as distinguem. A variante ingênua, `DSUNaive`, executa as operações sem empregar quaisquer heurísticas de otimização [GeeksforGeeks 2026]. Em contrapartida, a variante `DSUUnionByRank` incorpora o vetor auxiliar `rank[]` e redefine o método `union` de forma a fazer com que a raiz de menor posto seja subordinada à raiz de maior posto, conforme apresentado na Seção 4. A implementação correspondentes à variante `DSUNaive` é apresentada no Código 2.

Código 2: Naive `DSU`

```java
public class DSUNaive extends DSU {
    public DSUNaive(int n) {
        super(n);
    }

    @Override
    public void makeSet(int x) {
        parent[x] = x;
        operationCount++;
    }

    @Override
    public int findSet(int x) {
        while (parent[x] != x) {
            operationCount++;
            x = parent[x];
        }
        operationCount++;
        return x;
    }

    @Override
    public void union(int x, int y) {
        int rootX = findSet(x);
        int rootY = findSet(y);
        if (rootX != rootY) {
            parent[rootY] = rootX;
            operationCount++;
        }
    }

    @Override
    public String getName() {
        return "Naive";
    }
}
```

No Código 3. apresenta a implementação da variante `DSUUnionByRank`. Entre os aspectos mais relevantes dessa implementação, destaca-se a política de atualização do vetor `rank[]`, a qual constitui o elemento central da heurística *Union by Rank*. Para sua adequada compreensão, torna-se necessário explicitar em que circunstâncias o posto de uma raiz deve ser incrementado.

Código 3: Union By Rank `DSU`

```java
public class DSUUnionByRank extends DSU {
    protected int[] rank;

    public DSUUnionByRank(int n) {
        super(n);
    }

    @Override
    public void makeSet(int x) {
        parent[x] = x;
        if (rank == null) rank = new int[n];
        rank[x] = 0;
        operationCount++;
    }

    @Override
    public int findSet(int x) {
        while (parent[x] != x) {
            operationCount++;
            x = parent[x];
        }
        operationCount++;
        return x;
    }

    @Override
    public void union(int x, int y) {
        int rootX = findSet(x);
        int rootY = findSet(y);
        if (rootX == rootY) return;

        operationCount++;
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
    }

    @Override
    public String getName() {
        return "UnionByRank";
    }
}
```

Na implementação da heurística *Union by Rank*, o incremento do posto [LeetCode The Hard Way 2026] ocorre exclusivamente quando as duas raízes envolvidas na operação de união possuem o mesmo posto. Esse critério decorre da definição de posto adotada neste trabalho, segundo a qual o posto de um nó x representa um limite superior para a altura da árvore enraizada em x, conforme descrito por Cormen et al. [Cormen et al. 2009]. A justificativa teórica para essa escolha é apresentada na Seção 4.

Por sua vez, a variante `DSUFullTarjan`, descrito no Código 4, estende diretamente `DSUUnionByRank` e redefine apenas o método `findSet`, de modo a incorporar a heurística de compressão de caminho [CP-Algorithms 2026]. Tal relação de herança reproduz a própria organização conceitual dos algoritmos, visto que o *Full Tarjan* não se configura como uma implementação autônoma, mas como um aprimoramento incremental do *Union by Rank*, conforme descrito por Cormen et al. [Cormen et al. 2009].

Código 4: Full Tarjan `DSU`

```java
public class DSUFullTarjan extends DSUUnionByRank {
    public DSUFullTarjan(int n) {
        super(n);
    }

    @Override
    public int findSet(int x) {
        operationCount++;
        if (parent[x] != x) {
            parent[x] = findSet(parent[x]);
        }
        return parent[x];
    }

    @Override
    public String getName() {
        return "FullTarjan";
    }
}
```

3.2. Ambiente de Execução

Para garantir que os testes possam ser reproduzidos por outras pessoas, detalhamos aqui o *setup* utilizado nos experimentos.Todo o código-fonte e as instruções de como rodar o projeto estão disponíveis no GitHub[1]. A máquina onde o código rodou é equipada com um processador Intel Core i5-1135G7 (11ª geração), rodando a 2.40 GHz com 8 *threads* em arquitetura de 64 bits (*x86_64*). O sistema possui 7,5 GiB de memória RAM livre e 3,7 GiB de *swap*, e o sistema operacional base é o Arch Linux. Os *prints* do terminal comprovando esse hardware e memória estão, respectivamente, nas Figuras 7 e 8 do Apêndice.

[1] `https://github.com/Athoosz/FPAA-TP1-DSU`

Na parte de software, executamos a bateria de testes usando o OpenJDK versão 21.0.6 (LTS), especificamente a *build* Temurin 21.0.6+7 operando em modo *mixed* (Figura 9). Como a forma que a JVM otimiza o código em tempo de execução impacta diretamente no tempo final, extraímos as principais *flags* do compilador usando o comando `PrintFlagsFinal`. A Listagem 1 mostra os parâmetros ativos durante os testes (com o *print* original na Figura 10 do Apêndice). Isso documenta exatamente como o JIT foi configurado para compilar nossas variantes de DSU.

```
bool AlwaysCompileLoopMethods         = false
intx CICompilerCount                  = 4
bool CICompilerCountPerCPU            = true
intx CompileThreshold                = 10000
double CompileThresholdScaling        = 1.000000
intx Tier3CompileThreshold            = 2000
intx Tier4CompileThreshold            = 15000
bool UseCompiler                      = true
bool UseDynamicNumberOfCompilerThreads = true
```

Listing 1. Principais flags de compilação da JVM ativas no momento dos testes.

3.3. Protocolo de Medição

A infraestrutura destinada à avaliação experimental foi estruturada a partir de duas classes centrais, denominadas `OperationTime` e `OperationTimeResult`, cujas responsabilidades foram definidas de maneira complementar. A classe `OperationTime` é encarregada da execução dos experimentos, sendo responsável por aplicar, sobre uma instância de `DSU`, uma sequência controlada de operações e registrar as métricas de interesse. Em contrapartida, a classe `OperationTimeResult` destina-se exclusivamente ao encapsulamento dos dados coletados em cada execução, reunindo, em uma única estrutura, a identificação da variante analisada, os parâmetros experimentais adotados, o tempo medido e a contagem de acessos observada. Essa divisão foi adotada com o objetivo de preservar a coesão entre responsabilidades e reduzir o acoplamento entre a lógica de execução dos testes e a representação de seus resultados. Como consequência, a solução torna-se mais modular, extensível e adequada à realização de comparações experimentais reprodutíveis. Os Códigos 5 e 6 apresentam as implementações das classes `OperationTime` e `OperationTimeResult`, respectivamente.

3.4. O problema do JIT da JVM

Diferentemente de linguagens tradicionalmente compiladas, como C e C++, programas em Java não são convertidos diretamente para código de máquina no momento da compilação. Em vez disso, o código-fonte é inicialmente traduzido para *bytecode*, cuja execução é realizada pela *Java Virtual Machine* (JVM). Durante esse processo, a JVM pode inicialmente interpretar o *bytecode* e, à medida que determinados métodos passam a ser invocados com maior frequência, o compilador *Just-In-Time* (JIT) os converte para código nativo, significativamente mais eficiente em termos de desempenho [OpenJDK 2026].

Do ponto de vista experimental, essa característica introduz um fator de interferência relevante nas medições de tempo de execução. Como as primeiras invocações

Código 5: Classe `OperationTime`

```java
public class OperationTime {
    private final Random random = new Random(10);

    public OperationTimeResult run(DSU dsu, int n, int m) {
        dsu.resetOperationCount();

        for (int i = 0; i < Math.min(1000, m / 10); i++) {
            int a = random.nextInt(n);
            int b = random.nextInt(n);
            dsu.union(a, b);
        }

        dsu.resetOperationCount();

        long start = System.nanoTime();

        for (int i = 0; i < m; i++) {
            int a = random.nextInt(n);
            int b = random.nextInt(n);
            if (i % 3 == 0) {
                dsu.findSet(a);
            } else {
                dsu.union(a, b);
            }
        }

        long elapsed = System.nanoTime() - start;

        return new OperationTimeResult(
            dsu.getName(), n, m, elapsed,
            dsu.getOperationCount()
        );
    }
}
```

Código 6: Classe `OperationTimeResult`

```java
public class OperationTimeResult {
    public final String dsuName;
    public final int n;
    public final int m;
    public final long timeNanos;
    public final long operationCount;

    public OperationTimeResult(String dsuName, int n,
                int m, long timeNanos, long opCount) {
        this.dsuName = dsuName;
        this.n = n;
        this.m = m;
        this.timeNanos = timeNanos;
        this.operationCount = opCount;
    }

    @Override
    public String toString() {
        return String.format(
            "%-15s | n=%-8d | m=%-8d | tempo=%8.2f ms
            | ops=%d",
            dsuName, n, m, timeNanos / 1_000_000.0,
                operationCount
        );
    }
}
```

de um método tendem a ocorrer ainda em modo interpretado, elas apresentam, em geral, desempenho inferior ao das execuções subsequentes, já beneficiadas pela compilação JIT. Assim, caso o tempo fosse medido desde a primeira execução, a variante avaliada inicialmente poderia sofrer uma penalização artificial, comprometendo a validade da comparação entre as implementações.

Com o objetivo de mitigar esse efeito, o método `run` da classe `OperationTime` realiza, antes do início da medição, uma etapa de aquecimento. Nessa etapa, é criada uma instância independente do mesmo tipo de `DSU` sob avaliação, sobre a qual é executado um conjunto preliminar de operações. Esse procedimento tem como finalidade induzir a compilação dos métodos relevantes pelo JIT antes do acionamento do cronômetro, de modo que a medição reflita de forma mais fiel o desempenho efetivo da implementação.

A utilização de uma instância separada para o aquecimento constitui uma decisão essencial para a integridade do experimento. Caso essa etapa fosse executada sobre a mesma instância posteriormente utilizada na medição, o estado interno da estrutura, em especial os vetores `parent[]` e `rank[]`, já se encontraria alterado antes do início da coleta. Isso faria com que as diferentes variantes não partissem de condições equivalentes, comprometendo a comparabilidade dos resultados obtidos.

3.5. Captura do tempo

O tempo de execução foi mensurado por meio do método `System.nanoTime()`, que fornece a leitura de um relógio de alta resolução em nanossegundos. Tal método é mais adequado do que `System.currentTimeMillis()` para a medição de intervalos temporais, uma vez que não depende de ajustes aplicados ao relógio do sistema operacional, como aqueles decorrentes de sincronizações via NTP, os quais podem introduzir descontinuidades durante a coleta. Desse modo, o intervalo considerado na análise corresponde exclusivamente à execução das m operações sobre a estrutura `DSU`, desconsiderando tanto a etapa de aquecimento quanto a fase de inicialização da estrutura [Oracle 2026].

3.6. Sequência de operações e reprodutibilidade

O método `run` executa m operações sobre a instância de `DSU` recebida, sendo aproximadamente dois terços dessas operações compostos por chamadas a `union` e um terço por chamadas a `findSet`. Tal proporção foi adotada por representar um cenário plausível de utilização da estrutura, como aquele observado no algoritmo de Kruskal, no qual arestas processadas demandam consultas de representatividade por meio de `findSet` e, quando necessário, operações de fusão por meio de `union` [Cormen et al. 2009].

A reprodutibilidade dos experimentos é assegurada pelo uso de uma semente fixa no gerador de números pseudoaleatórios, implementado por meio de `new Random(10)`. Essa escolha garante que as três variantes da estrutura recebam exatamente a mesma sequência de valores para a e b em cada configuração de teste. Desse modo, eventuais diferenças observadas nos resultados podem ser atribuídas ao comportamento intrínseco de cada implementação, e não a variações na entrada utilizada.

Adicionalmente, o contador de acessos à memória, mantido na classe abstrata `DSU` por meio do atributo `operationCount`, é reinicializado mediante a chamada ao método `resetOperationCount()` imediatamente antes do início de cada medição.

Com isso, garante-se que operações executadas em etapas anteriores, inclusive durante o aquecimento da JVM, não sejam incorporadas à coleta corrente, preservando a consistência e a comparabilidade dos resultados experimentais.

3.7. Separação entre medição e armazenamento

Os dados produzidos por cada execução experimental são encapsulados pela classe `OperationTimeResult`, a qual armazena a identificação da variante avaliada, os parâmetros n e m, o tempo de execução medido em nanossegundos e a contagem total de acessos registrada pelo contador interno da estrutura `DSU`. A separação entre a lógica de medição, concentrada na classe `OperationTime`, e o armazenamento dos resultados, atribuído à classe `OperationTimeResult`, segue o princípio da separação de responsabilidades. Essa organização favorece a modularidade do projeto, uma vez que permite que a camada encarregada da geração dos comparativos, implementada na classe `Main`, seja modificada ou substituída sem a necessidade de alterações na infraestrutura de medição. Tal escolha mostra-se particularmente relevante para a extensibilidade da solução, considerando a possibilidade de adoção de diferentes formatos de saída ao longo da análise experimental.

4. Fundamentação Teórica

4.1. Problema resolvido pela estrutura DSU

A estrutura DSU (Disjoint Set Union) é utilizada para representar e manipular uma coleção de conjuntos disjuntos dinâmicos. Considere um universo com n elementos, em que, inicialmente, cada elemento pertence a um conjunto isolado. Ao longo da execução, alguns desses conjuntos podem ser unidos, formando grupos maiores. Nesse contexto, é necessário responder de forma eficiente a duas operações principais: determinar a qual conjunto um elemento pertence e verificar se dois elementos pertencem ao mesmo conjunto.

Esse problema é tratado na literatura, como em Cormen et al., no contexto de conectividade dinâmica, em que se mantém uma coleção de conjuntos disjuntos dinâmicos, denotada por $S = \{S_1, S_2, \ldots, S_k\}$. Cada conjunto é identificado por um representante, isto é, um de seus próprios elementos, utilizado como referência para identificar o grupo ao qual os demais pertencem.

4.2. As três operações fundamentais

Conforme descrito por Cormen et al. na Seção 21.1, a estrutura DSU é baseada em três operações fundamentais:

- **MAKE-SET**(x): cria um novo conjunto cujo único membro, e portanto seu representante, é x. Essa operação exige que x ainda não pertença a nenhum conjunto existente.
- **UNION**(x, y): une em um único conjunto os conjuntos que contêm os elementos x e y. Após essa operação, os dois conjuntos originais deixam de existir separadamente.
- **FIND-SET**(x): retorna um ponteiro para o representante do conjunto ao qual o elemento x pertence.

O custo das operações é analisado em função de dois parâmetros: n, que representa o número de operações **MAKE-SET** realizadas, isto é, o número total de elementos, e m, que representa o número total de operações executadas.

4.3. Representação por listas ligadas e custo da operação

Uma das formas mais simples de implementar a estrutura de conjuntos disjuntos consiste em representar cada conjunto por meio de uma lista ligada. Nessa abordagem, cada conjunto possui um objeto com ponteiros *head* e *tail*, que indicam, respectivamente, o primeiro e o último elemento da lista. Além disso, cada nó da lista armazena o elemento correspondente, um ponteiro para o próximo nó e um ponteiro de volta para o objeto do conjunto. O representante do conjunto é definido como o elemento armazenado no primeiro nó da lista.

Com essa representação, as operações **MAKE-SET** e **FIND-SET** podem ser executadas em tempo constante. A operação **MAKE-SET(**x**)** cria uma nova lista contendo apenas o elemento x, enquanto **FIND-SET(**x**)** apenas segue o ponteiro de x até o objeto do conjunto e retorna o representante apontado por *head*. Assim, ambas têm custo $O(1)$.

A principal limitação dessa representação está na operação **UNION(**x, y**)**. Na implementação mais direta, a lista do conjunto que contém y é anexada ao final da lista do conjunto que contém x. Embora o ponteiro *tail* permita localizar rapidamente o ponto de anexação, essa operação exige a atualização do ponteiro de volta ao conjunto para todos os nós que estavam originalmente na lista de y. Dessa forma, o custo de uma operação **UNION** é linear no tamanho da lista anexada.

Esse comportamento permite a construção de um caso desfavorável em que o custo total se torna quadrático. Suponha inicialmente n operações **MAKE-SET**, criando os conjuntos $\{x_1\}, \{x_2\}, \ldots, \{x_n\}$. Em seguida, realizam-se $n-1$ operações de união de modo que, em cada etapa, um conjunto já grande seja passado como segundo argumento da operação, isto é, justamente como a lista que será anexada e terá seus ponteiros atualizados. Assim, na primeira união, atualiza-se 1 ponteiro; na segunda, 2 ponteiros; na terceira, 3 ponteiros; e assim sucessivamente, até a última união, que atualiza $n-1$ ponteiros.

Consequentemente, o número total de atualizações realizadas pelas operações de união é dado por

$$\sum_{i=1}^{n-1} i = \Theta(n^2).$$

Como o número total de operações é $m = n + (n-1) = 2n - 1$, conclui-se que o custo amortizado por operação é da ordem de $\Theta(n)$. Portanto, embora a representação por listas ligadas seja simples e torne eficientes as operações **MAKE-SET** e **FIND-SET**, sua implementação ingênua para **UNION** apresenta desempenho insatisfatório para valores grandes de n.

4.4. Floresta de conjuntos disjuntos sem heurísticas (*Naive*)

Na representação por floresta, cada elemento x armazena apenas um ponteiro para seu pai $x.p$. A raiz de cada árvore é o representante do conjunto e aponta para si mesma.

A operação **MAKE-SET**(x) inicializa $x.p = x$ em tempo $O(1)$. A operação **FIND-SET**(x) percorre a cadeia de pais até encontrar a raiz, isto é, o nó tal que $x.p = x$. No pior caso, quando uniões sucessivas produzem uma cadeia linear de n nós, essa operação exige $O(n)$ passos.

A operação **UNION**(x, y) invoca **FIND-SET** duas vezes para localizar as raízes e então faz uma raiz apontar para a outra. Sem critério de balanceamento, uma sequência desfavorável de uniões pode degradar a estrutura a uma corrente, fazendo com que operações subsequentes de **FIND-SET** percorram $O(n)$ nós. É justamente esse comportamento degenerado que motiva as heurísticas apresentadas nas seções seguintes.

4.5. Florestas de conjuntos disjuntos (*Union by Rank*)

Para contornar a degradação linear descrita na seção anterior, Cormen et al. propõem a heurística de *união pelo posto* (*union by rank*) que nessa estratégia, associa-se a cada raiz um valor chamado *posto* (*rank*), que funciona como um limite superior para a altura da árvore. Inicialmente, em cada operação **MAKE-SET**, cria-se uma árvore com um único nó, cujo posto é igual a 0.

Durante a execução de **UNION**(x, y), comparam-se os postos das raízes dos conjuntos que contêm x e y. Se os postos forem diferentes, a raiz de menor posto passa a apontar para a raiz de maior posto, e nenhum posto é alterado. Se os postos forem iguais, uma das raízes é escolhida arbitrariamente para tornar-se pai da outra, e seu posto é incrementado em uma unidade.

Essa heurística é conceitualmente próxima da união pela altura ou pela quantidade de nós da árvore, pois em todos os casos o objetivo é evitar o crescimento excessivo da profundidade. No caso específico da união pelo posto, o balanceamento obtido garante que a altura das árvores permaneça limitada por $O(\log n)$ quando considerada isoladamente, isto é, sem compressão de caminho. Como consequência, a operação **FIND-SET** percorre, no pior caso, no máximo um número logarítmico de nós.

Assim, a união pelo posto constitui uma estratégia de balanceamento que melhora significativamente o desempenho da estrutura de conjuntos disjuntos, servindo de base para a obtenção de versões ainda mais eficientes quando combinada com a heurística de compressão de caminho.

4.6. Compressão de caminho e análise da complexidade amortizada (*Full Tarjan*)

Além da heurística de união pelo posto, Cormen et al. apresentam a *compressão de caminho* (*path compression*) como uma segunda estratégia para melhorar a eficiência da estrutura de conjuntos disjuntos. Essa heurística é aplicada durante a operação **FIND-SET**. Em vez de apenas percorrer os ponteiros de pai até alcançar a raiz da árvore, a operação também reorganiza os ponteiros dos nós visitados, fazendo com que todos eles passem a apontar diretamente para a raiz encontrada.

Em termos intuitivos, a compressão de caminho não altera a partição dos conjuntos, tampouco modifica o representante de cada conjunto; ela apenas reduz a profundidade dos nós acessados. Dessa forma, embora uma chamada isolada de **FIND-SET** ainda possa percorrer um caminho relativamente longo, as chamadas subsequentes tendem a tornar-se muito mais rápidas, pois a árvore vai sendo progressivamente "achatada".

Na implementação recursiva clássica, essa operação ocorre em duas etapas. Primeiro, segue-se a cadeia de pais até a raiz. Em seguida, durante o retorno da recursão, cada nó do caminho visitado tem seu ponteiro de pai atualizado para apontar diretamente para essa raiz. Assim, a compressão de caminho atua como uma forma de reorganização local da estrutura, reduzindo o custo de buscas futuras sem comprometer a corretude da representação.

Quando a compressão de caminho é utilizada em conjunto com a heurística de união pelo posto, obtém-se o melhor resultado assintótico para a estrutura DSU. A ideia central é que a união pelo posto evita o crescimento excessivo da altura das árvores, enquanto a compressão de caminho reduz drasticamente os comprimentos dos caminhos efetivamente percorridos nas operações de busca. Em conjunto, essas duas heurísticas impedem que a estrutura se degrade de forma significativa, mesmo ao longo de sequências extensas de operações.

Cormen et al. demonstram que, para uma sequência de m operações sobre n elementos, o custo total é limitado por

$$O(m\,\alpha(n)),$$

em que $\alpha(n)$ denota a inversa da função de Ackermann. Embora a função de Ackermann cresça extremamente rápido, sua inversa cresce extremamente devagar. Como consequência, $\alpha(n)$ assume valores muito pequenos para qualquer entrada de interesse prático. Na prática, esse fator é tão lento que pode ser tratado como praticamente constante.

Desse modo, a combinação entre união pelo posto e compressão de caminho torna a estrutura de conjuntos disjuntos extremamente eficiente. Ainda que a demonstração formal desse resultado envolva análise amortizada e propriedades específicas dos postos das árvores, a intuição principal é simples: evita-se a formação de árvores desbalanceadas e, ao mesmo tempo, encurtam-se continuamente os caminhos percorridos pelas operações **FIND-SET**. Isso explica por que a estrutura DSU é amplamente utilizada em algoritmos que exigem manutenção dinâmica de conectividade, como o algoritmo de Kruskal.

5. Discussão de Resultados

Os experimentos foram conduzidos em três conjuntos distintos de configurações, estabelecidos de forma específica para cada variante, em virtude das diferenças de viabilidade computacional entre as implementações analisadas. Os resultados correspondentes são apresentados nas Figuras 1 a 5 e discutidos a seguir. No Apêndice, é apresentada a Figura 6 das saídas geradas no terminal durante a execução dos experimentos, com a finalidade de documentar os dados brutos coletados e oferecer suporte complementar à análise dos resultados discutidos ao longo do trabalho.

5.1. Análise da Variante Naive

A Figura 2 apresenta, em escala logarítmica, o tempo de execução das três variantes nos pontos em que todas puderam ser avaliadas simultaneamente, a saber, $n = 1.000$, $n = 10.000$ e $n = 50.000$. Mesmo nessa escala, observa-se de forma clara a inclinação significativamente mais acentuada da curva correspondente à variante *Naive* em comparação às

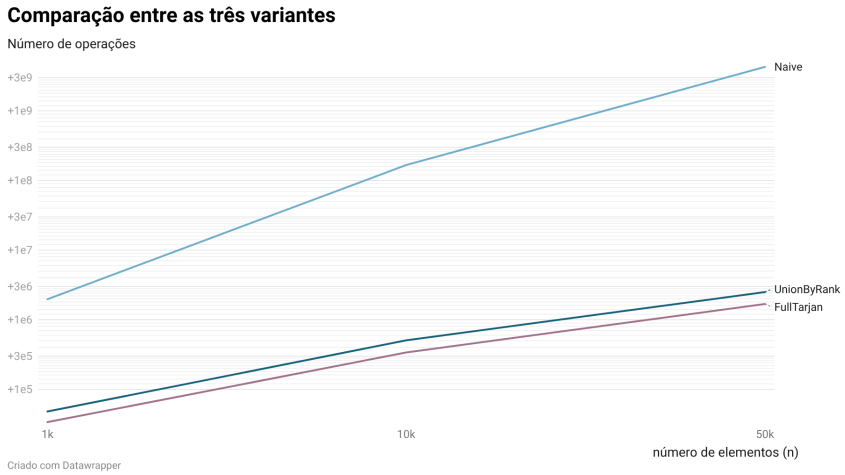

Figura 1. Tempo de execução em função do número de elementos n para as três variantes do DSU, em escala logarítmica. O Naive foi testado até n=50.000 devido à inviabilidade computacional para valores maiores.

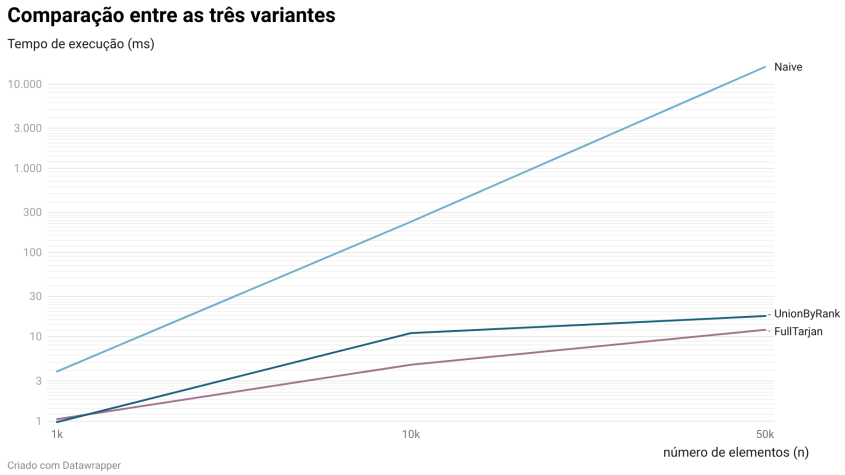

Figura 2. Número de acessos à memória em função de n para as três variantes, em escala logarítmica. A distância entre o Naive e as variantes otimizadas evidencia a diferença entre as classes de complexidade $O(n^2)$ e $O(m \log n)$.

demais. Enquanto as curvas de `UnionByRank` e `FullTarjan` permanecem próximas à porção inferior do gráfico, a curva da variante *Naive* apresenta crescimento expressivo a cada incremento de n. A Figura 1, por sua vez, apresenta a mesma comparação sob a perspectiva do número de operações ou acessos à memória, confirmando o mesmo padrão de comportamento.

A análise quantitativa dos valores apresentados na Figura 2 reforça essa evidência visual. Quando n cresce de 1.000 para 10.000, isto é, por um fator de 10, o tempo de execução da variante *Naive* aumenta de $3{,}87$ ms para $232{,}21$ ms, o que corresponde a aproximadamente 60 vezes mais. Na transição de $n = 10.000$ para $n = 50.000$, equivalente a um fator de 5, o tempo cresce de $232{,}21$ ms para $15.998{,}38$ ms, representando aproximadamente 69 vezes mais. Verifica-se, portanto, que o crescimento do tempo de execução supera sistematicamente o crescimento da entrada, o que constitui uma evidência empírica de comportamento superlinear. Tal resultado é consistente com a análise apresentada por Cormen et al. [Cormen et al. 2009], segundo a qual, na ausência de heurísticas de balanceamento, a floresta pode degenerar em correntes lineares, tornando a operação `findSet` proporcional a $O(n)$ no pior caso.

Os resultados da Figura 2 conduzem à mesma conclusão. Para $n = 50.000$, a contagem de acessos à memória ultrapassa $4{,}3$ bilhões ao longo de 500.000 operações, o que corresponde a uma média aproximada de 8.600 acessos por operação. Esse valor mostra-se compatível com um custo proporcional a n por operação, em conformidade com o comportamento esperado para uma implementação sem mecanismos de balanceamento.

A inviabilidade da variante *Naive* para valores superiores de n constitui, em si, um resultado experimental relevante. Tal limitação evidencia que a ausência de heurísticas de otimização torna essa implementação impraticável em aplicações de média e grande escala, como aquelas que envolvem o algoritmo de Kruskal aplicado a grafos com milhões de arestas.

5.2. Variante Union by Rank

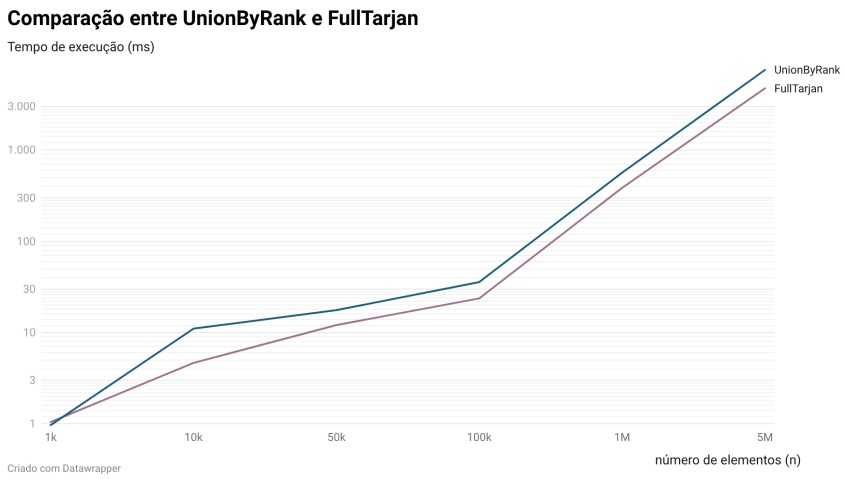

Figura 3. Tempo de execução das variantes otimizadas de n=1.000 até n=5.000.000, em escala logarítmica. A distância crescente entre as curvas evidencia o ganho da compressão de caminho em entradas maiores.

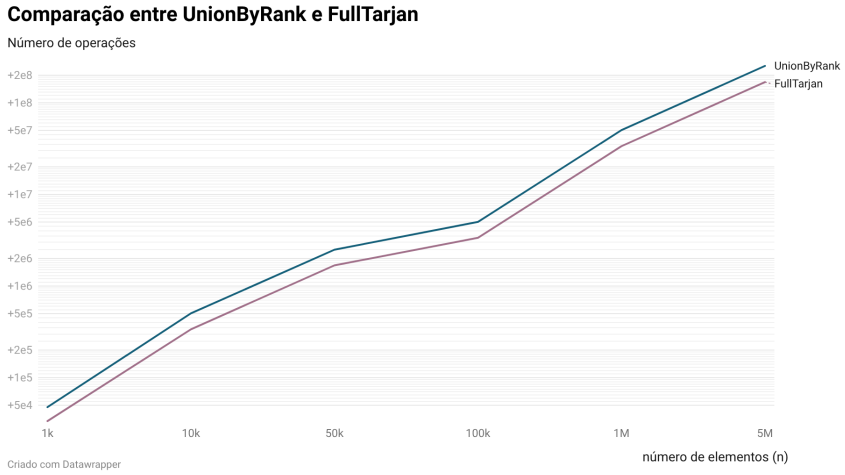

Figura 4. Número de acessos à memória das variantes otimizadas de $n = 1.000$ até $n = 5.000.000$, em escala logarítmica. A separação constante entre as curvas é consistente com a redução estável de aproximadamente 33% de acessos observada em todos os tamanhos testados.

A Figura 3 apresenta, em escala logarítmica, o tempo de execução das variantes `UnionByRank` e `FullTarjan` no intervalo compreendido entre $n = 1.000$ e $n = 5.000.000$. Considerando, inicialmente, a variante `UnionByRank`, observa-se que sua curva apresenta inclinação substancialmente mais suave do que aquela verificada para a variante *Naive* na Figura 2, evidenciando a mudança de comportamento assintótico proporcionada pela heurística de união pelo posto.

A análise dos fatores de crescimento do tempo de execução da variante `UnionByRank`, visíveis na Figura 3, reforça essa interpretação. Quando n cresce de 1.000 para 10.000, isto é, por um fator de 10, o tempo aumenta de $0{,}97$ ms para $11{,}06$ ms, correspondendo a aproximadamente $11{,}4$ vezes. Na transição de $n = 10.000$ para $n = 100.000$, também com crescimento de 10 vezes na entrada, observa-se um fator de aumento de aproximadamente $3{,}2$. De $n = 100.000$ para $n = 1.000.000$, novamente por um fator de 10, o tempo cresce cerca de $15{,}8$ vezes. Por fim, de $n = 1.000.000$ para $n = 5.000.000$, isto é, por um fator de 5, registra-se um aumento de aproximadamente $13{,}3$ vezes. Embora tais fatores apresentem oscilações, não se observa uma tendência consistente de aceleração, o que é compatível com o comportamento $O(m \log n)$ atribuído a essa variante por Cormen et al. [Cormen et al. 2009]. As variações observadas são esperadas em medições temporais realizadas em ambiente JVM, uma vez que fatores como escalonamento do sistema operacional e otimizações realizadas pelo compilador *Just-In-Time* podem introduzir flutuações que não se manifestam com a mesma intensidade em métricas estruturais.

A Figura 4 apresenta a evolução da contagem de acessos à memória da variante `UnionByRank` no mesmo intervalo de valores de n. Essa métrica confirma o comportamento observado de forma mais estável: a contagem cresce de 47.710 acessos, para $n = 1.000$, para $252.213.983$ acessos, em $n = 5.000.000$, o que representa um aumento aproximado de 5.285 vezes para um crescimento de 5.000 vezes no tamanho da entrada. Tal resultado é coerente com um crescimento próximo ao linear no número total de operações executadas, compatível com o comportamento $O(m \log n)$ quando m cresce

proporcionalmente a n.

Esses resultados confirmam, portanto, que a heurística de união pelo posto é suficiente para eliminar o comportamento degenerado observado na variante Naive, reduzindo o crescimento para $O(m \log n)$. Contudo, como será evidenciado na seção seguinte, essa heurística isolada ainda deixa margem para ganhos adicionais quando combinada com a compressão de caminho.

5.3. Variante Full Tarjan

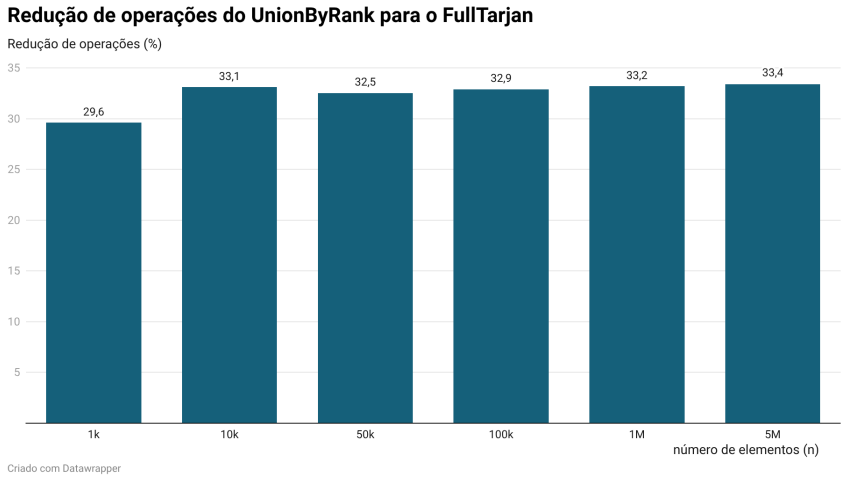

Figura 5. Redução percentual de acessos à memória do `UnionByRank` para o `FullTarjan`, calculada pela equação $R(n) = ((O_{UBR} - O_{FT})/O_{UBR}) \times 100$. A estabilidade de $R(n)$ em torno de 33% confirma o comportamento amortizado constante $O(\alpha(n))$ descrito por Cormen et al. [Cormen et al. 2009], Seção 21.4.

A curva correspondente à variante `FullTarjan`, apresentada na Figura 3, acompanha a tendência geral observada para a variante `UnionByRank`, porém permanece consistentemente abaixo desta em todos os pontos avaliados. Como ambas as variantes compartilham a mesma implementação da operação `union`, essa diferença pode ser atribuída exclusivamente ao efeito da heurística de compressão de caminho incorporada ao `FullTarjan`. Para a maior entrada testada, $n = 5.000.000$, a variante `FullTarjan` registrou tempo de execução de $4.741{,}19$ ms, em contraste com $7.545{,}54$ ms obtidos pela variante `UnionByRank`, o que corresponde a uma redução aproximada de 37% no tempo de execução.

A Figura 4 permite observar o mesmo fenômeno sob a perspectiva da contagem de acessos à memória, métrica menos suscetível a oscilações introduzidas pelo ambiente de execução. Verifica-se que a distância entre as curvas das duas variantes é visível e relativamente constante ao longo de toda a faixa de valores de n considerada, o que indica que o benefício proporcionado pela compressão de caminho não se restringe a casos pontuais, mas se manifesta de forma recorrente em todos os tamanhos de entrada avaliados.

Com o objetivo de quantificar diretamente esse ganho, a Figura 5 apresenta a redução percentual no número de acessos à memória, calculada de acordo com a Equação 1.

$$R(n) = \frac{O_{UBR}(n) - O_{FT}(n)}{O_{UBR}(n)} \times 100 \tag{1}$$

em que $O_{UBR}(n)$ representa a quantidade de acessos observada na variante `UnionByRank` para uma entrada de tamanho n, enquanto $O_{FT}(n)$ representa a quantidade correspondente na variante `FullTarjan`.

O resultado mais relevante evidenciado pela Figura 5 não reside no valor absoluto de $R(n)$, mas em sua estabilidade ao longo da escala analisada. Partindo de $29{,}6\%$ em $n = 1.000$, a redução converge para aproximadamente 33% a partir de $n = 10.000$, mantendo-se nesse patamar até $n = 5.000.000$, sem tendência expressiva de crescimento ou de queda. Tal comportamento constitui uma evidência empírica compatível com o custo amortizado $O(\alpha(n))$ descrito por Cormen et al. [Cormen et al. 2009].

A interpretação desse resultado é direta. Caso a compressão de caminho se tornasse progressivamente mais eficaz à medida que o tamanho da entrada aumentasse, seria esperado que $R(n)$ apresentasse crescimento monotônico na Figura 5. De forma análoga, caso sua eficácia relativa diminuísse para entradas maiores, a tendência observada seria de redução de $R(n)$. A estabilidade empírica verificada indica, portanto, que a compressão de caminho mantém eficácia relativa aproximadamente constante, independentemente da escala da entrada. Esse comportamento é consistente com a natureza extremamente lenta de crescimento da função $\alpha(n)$, inversa da função de Ackermann, a qual, para valores de n de interesse prático, assume valores muito pequenos, conforme discutido por Cormen et al. [Cormen et al. 2009].

6. Conclusão

Este estudo investigou o impacto da adoção de estruturas de dados otimizadas na eficiência de algoritmos de conectividade dinâmica, especificamente no contexto da estrutura *Disjoint Set Union* (DSU). Do ponto de vista quantitativo, o achado mais significativo foi a drástica redução do custo computacional ao integrar as heurísticas de Robert Tarjan, transformando um comportamento compatível com $O(n^2)$ na variante *Naive* em um desempenho de tempo amortizado quase constante na versão *Full Tarjan*.

Conforme evidenciado pela comparação de acessos à memória na Figura 2, o diferencial do método encontra-se na estabilidade da redução de operações. O gráfico detalhado na Figura 5 demonstra que esse ganho percentual converge e se mantém constante em aproximadamente 33% para entradas massivas. Esse comportamento constitui uma evidência empírica robusta da eficácia da compressão de caminho e prova a sua aderência à complexidade assintótica $O(m\alpha(n))$, corroborando a análise teórica de Cormen et al. [Cormen et al. 2009].

Do ponto de vista da engenharia de software, a inviabilidade da implementação ingênua para amostras superiores a 50.000 elementos ressalta que a escolha da estrutura de dados não é apenas uma questão de otimização periférica, mas um requisito de viabilidade para sistemas reais e escaláveis. A modularização adotada no projeto, referenciada na estrutura da classe abstrata do Código 1, permitiu validar que o domínio sobre a complexidade amortizada possibilita que problemas de infraestrutura de redes e bioinformática sejam processados com alta performance, independentemente do crescimento massivo do volume de dados.

Em suma, a transição de um custo de execução proibitivo para uma resposta quase instantânea, garantida pelas heurísticas de união por posto e compressão de caminho, reforça a importância da análise algorítmica no ciclo de vida do desenvolvimento. Conclui-se que a reflexão profunda sobre a adequação das estruturas de dados no projeto de algoritmos reais, sustentada por métricas determinísticas de acesso à memória, é o que garante que a infraestrutura de software permaneça responsiva e eficiente sob condições de estresse, assegurando a entrega de soluções robustas para a indústria.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3 edition.

CP-Algorithms (2026). Disjoint set union. `https://cp-algorithms.com/data_structures/disjoint_set_union.html`. Acesso em: abr. 2026.

GeeksforGeeks (2026). Introduction to disjoint set (union-find data structure). `https://www.geeksforgeeks.org/dsa/introduction-to-disjoint-set-data-structure-or-union-find-algorithm/`. Acesso em: abr. 2026.

LeetCode The Hard Way (2026). Disjoint set union (dsu). `https://leetcodethehardway.com/tutorials/graph-theory/disjoint-set-union`. Acesso em: abr. 2026.

OpenJDK (2026). Hotspot runtime overview. `https://openjdk.org/groups/hotspot/docs/RuntimeOverview.html`. Acesso em: 16 abr. 2026.

Oracle (2026). Java se documentation: System.nanotime(). `https://docs.oracle.com/en/java/docs/api/java.base/java/lang/System.html`. Acesso em: 16 abr. 2026.

Apêndice

```
Variante        | n           | m           | Tempo (ms)      | Operações
-----------------------------------------------------------------------------
=== Naive ===
Naive           | n=1000      | m=10000     | tempo=    3.87 ms | ops=1964970
Naive           | n=10000     | m=100000    | tempo=  232.21 ms | ops=167279619
Naive           | n=50000     | m=500000    | tempo=15998.38 ms | ops=4307096947

=== UnionByRank e FullTarjan ===
UnionByRank     | n=1000      | m=10000     | tempo=    0.97 ms | ops=47710
FullTarjan      | n=1000      | m=10000     | tempo=    1.05 ms | ops=33590
UnionByRank     | n=10000     | m=100000    | tempo=   11.06 ms | ops=503432
FullTarjan      | n=10000     | m=100000    | tempo=    4.66 ms | ops=337137
UnionByRank     | n=50000     | m=500000    | tempo=   17.61 ms | ops=2490570
FullTarjan      | n=50000     | m=500000    | tempo=   12.09 ms | ops=1680276
UnionByRank     | n=100000    | m=1000000   | tempo=   35.78 ms | ops=5008831
FullTarjan      | n=100000    | m=1000000   | tempo=   23.70 ms | ops=3358661
UnionByRank     | n=1000000   | m=10000000  | tempo=  564.82 ms | ops=50235136
FullTarjan      | n=1000000   | m=10000000  | tempo=  384.92 ms | ops=33577221
UnionByRank     | n=5000000   | m=50000000  | tempo= 7545.54 ms | ops=252213983
FullTarjan      | n=5000000   | m=50000000  | tempo= 4741.19 ms | ops=167883081
```

Figura 6. Resultados experimentais das três variantes do DSU para diferentes configurações de n e m, exibindo o tempo de execução em milissegundos e o número de acessos à memória registrados.

```
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 39 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          8
  On-line CPU(s) list:           0-7
Vendor ID:                       GenuineIntel
  Model name:                    11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
    CPU family:                  6
    Model:                       140
    Thread(s) per core:          2
    Core(s) per socket:          4
    Socket(s):                   1
    Stepping:                    1
    Microcode version:           0xbe
    CPU(s) scaling MHz:          16%
    CPU max MHz:                 4200.0000
    CPU min MHz:                 400.0000
    BogoMIPS:                    4838.40
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe sys
                                 all nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf tsc_kn
                                 wn_freq pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcn
                                  tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l2 cdp_l2 ssbd ibrs ibpb stibp
                                 brs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdt_a avx512f avx
                                 12dq rdseed adx smap avx512ifma clflushopt clwb intel_pt avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves s
                                 lit_lock_detect user_shstk dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp hwp_pkg_req vnmi avx512vbmi umip p
                                 u ospke avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid movdiri movdir64b fsrm avx512_vp
                                 intersect md_clear ibt flush_l1d arch_capabilities
Virtualization features:
  Virtualization:                VT-x
```

Figura 7. Especificações do processador e arquitetura do sistema obtidas via terminal.

```
[bruna@archlinux ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.5Gi       3.8Gi       1.2Gi       719Mi       3.4Gi       3.6Gi
Swap:          3.7Gi          0B       3.7Gi
```

Figura 8. Disponibilidade de memória RAM e swap do sistema.

```
[bruna@archlinux ~]$ java --version
openjdk 21.0.6 2025-01-21 LTS
OpenJDK Runtime Environment Temurin-21.0.6+7 (build 21.0.6+7-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.6+7 (build 21.0.6+7-LTS, mixed mode, sharing)
```

Figura 9. Versão do ambiente de execução Java (JVM) utilizado nos testes.

```
[bruna@archlinux ~]$ java -XX:+PrintFlagsFinal -version | grep -i compile
     bool AlwaysCompileLoopMethods                 = false                                   {product} {default}
     intx CICompilerCount                          = 4                                       {product} {ergonomic}
     bool CICompilerCountPerCPU                    = true                                    {product} {default}
 ccstrlist CompileCommand                          =                                         {product} {default}
    ccstr CompileCommandFile                       =                                         {product} {default}
 ccstrlist CompileOnly                             =                                         {product} {default}
     intx CompileThreshold                         = 10000                                {pd product} {default}
   double CompileThresholdScaling                  = 1.000000                                {product} {default}
      int CompilerThreadPriority                   = -1                                      {product} {default}
     intx CompilerThreadStackSize                  = 1024                                 {pd product} {default}
     bool DontCompileHugeMethods                   = true                                    {product} {default}
    uintx IncreaseFirstTierCompileThresholdAt      = 50                                      {product} {default}
     intx Tier2CompileThreshold                    = 0                                       {product} {default}
     intx Tier3CompileThreshold                    = 2000                                    {product} {default}
     intx Tier4CompileThreshold                    = 15000                                   {product} {default}
     intx TieredCompileTaskTimeout                 = 50                                      {product} {default}
     bool UseCompiler                              = true                                    {product} {default}
     bool UseDynamicNumberOfCompilerThreads        = true                                    {product} {default}
openjdk version "21.0.6" 2025-01-21 LTS
OpenJDK Runtime Environment Temurin-21.0.6+7 (build 21.0.6+7-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.6+7 (build 21.0.6+7-LTS, mixed mode, sharing)
```

Figura 10. Captura de tela demonstrando as principais flags de compilação da JVM ativas durante os experimentos.